PROJECT REPORT OF INDUSTRY ORIENTED HANDS ON EXPERIENCE (IOHE)

ON

CaloryInsight

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Deepak (2210991479)
Gunnidhi (2210991592)
Anurag Kumar (2210991320)

Supervised By:

Mentor
Dr. Rajat Takkar
Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA

CONTENTS

S.No.	Title	Page No.
	Declaration	3
	Acknowledgement	4
	Abstract	5
1	Introduction	6-8
2	Methodology	8-9
3	Tools and Technologies	10
4	Implementation	11
5	System Design and Architecture	11-12
6	Workflow	13-14
7	Project Setup Guide	15
8	Major Findings and Outcomes	16
9	Screenshots and Code of Project	16-24
10	Conclusion and Future Scope	25
11	References	26
12	Appendices	25

DECLARATION

I hereby certify that the work which is being presented in the project report entitled **"CaloryInsight"** in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of Dr. Rajat Takkar. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura **Deepak**

Date: 13-05-2026 (2210991479)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Rajat Takkar
Assistant Professor
Department of Computer Science and Engineering
Chitkara University Institute of Engineering and Technology,
Chitkara University, Punjab, India

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported me in the successful completion of my project **"CaloryInsight"**. This project would not have been possible without the guidance, encouragement, and assistance of many individuals.

First and foremost, I would like to thank my teachers and mentors for their valuable guidance, continuous support, and constructive suggestions throughout the development of this project. Their knowledge and motivation helped me overcome challenges and improve my skills.

I would like to thank everyone who directly or indirectly contributed to the completion of **"CaloryInsight"**. This project has been a valuable learning experience, and I am grateful for all the support I received.

Abstract

In today's health-conscious world, keeping track of daily fitness goals can often feel confusing and overwhelming. Many existing tools are either overly complicated or provide inaccurate estimates of physical activity and calorie expenditure.

Built using the modern MERN architecture (MongoDB, Express, React, and Node.js), this application provides individuals with an easy way to track their wellness. Instead of generic guesswork, the system takes a user's personal stats—like weight, age, biological gender, and exercise type—and calculates calories burned using standard oxygen consumption metrics (the Metabolic Equivalent of Task formula).

In addition to activity logging, the platform features a straightforward WHO-compliant BMI calculator and a nutrition directory mapping over 50 everyday foods. Ultimately, CaloryInsight bridges the gap between raw health metrics and practical, actionable everyday health awareness.

1. Introduction

Physical wellness has evolved from a casual hobby into an essential daily priority. As people become more focused on weight management and overall health, there is an increasing demand for tools that help track physical output. However, many individuals struggle with accountability simply because modern tools feel scattered. Calculating workout results or searching for basic food data usually requires jumping between multiple programs, which quickly leads to user burnout.

To create a more consistent lifestyle, users need a seamless health ecosystem. “CaloryInsight” was developed to serve as a central hub for personal wellness. By combining backend efficiency with a modern, dynamic web interface, the system gives people instant access to vital biometric calculations.

The application utilizes the Metabolic Equivalent of Task (MET) formula—a methodology widely respected in sports science—to translate workout durations into accurate caloric burn rates. By accounting for personal metrics like age, weight, and biological gender, CaloryInsight delivers customized results. Backed by the scalable MERN tech stack, this project successfully demonstrates how streamlined digital interfaces can comfortably empower healthier human decisions.

1.1. Problem Statement

Despite widespread public awareness regarding the benefits of exercise and healthy eating, standard tracking methods often fail to meet an individual’s exact physical needs. Many existing systems are unable to provide accurate and personalized wellness guidance, which limits their effectiveness for users seeking better health outcomes.

One major issue is generalized calculations, where traditional wellness platforms use one-size-fits-all estimations for exercise metrics. These systems often calculate calorie burn rates in the same way for all users, ignoring important physiological differences such as age, body weight, metabolism, and other biological factors. As a result, the recommendations may not suit every individual.

Another challenge is information overload and fragmentation. Users frequently need to switch between multiple platforms to track workouts, calculate BMI, and research diet-related information. Managing several disconnected applications reduces convenience, wastes time, and often decreases user motivation to stay consistent.

In addition, prohibitive costs create another barrier for many people. Advanced fitness analytics and continuous health monitoring are commonly available only through expensive smartwatches or paid premium health applications. This makes it difficult for average users to access quality fitness tools.

1.2. Objective of the Project

The primary objective of CaloryInsight is to develop a smart, user-friendly, and efficient fitness web application that helps users monitor their health in a simple way. The system is designed to track daily calorie intake, calories burned, and overall fitness activities. It also provides BMI calculation to help users understand their health status and maintain a balanced lifestyle.

Another important objective of the project is to combine multiple health-related features into a single platform. Instead of using separate applications for calorie tracking, workout monitoring, and BMI analysis, users can access all these services through one dashboard. This improves convenience, saves time, and enhances user experience.

1.3. Outcome

The outcome of **CaloryInsight** is the successful development of a smart and user-friendly fitness web application that helps users manage their health efficiently. The system provides an integrated platform where users can track calorie intake, calories burned, BMI, and workout activities in one place.

The project successfully simplifies health monitoring by combining multiple features into a single dashboard. Users no longer need to depend on different applications for calorie tracking, BMI calculation, and workout monitoring, which improves convenience and saves time.

Another important outcome is the secure management of user data through the MERN stack architecture. User information, workout records, and calorie history are stored safely in the MongoDB database, while authentication features ensure privacy and security.

1.4. Key Features

- Secure user registration and login.
- BMI calculation system.
- Daily calorie intake tracking.
- Calories burned monitoring.
- Workout activity tracking.
- Health progress dashboard.
- Reports and analytics.

2. Methodology

2.1 Agile Development Methodology:

2.1.1. Planning

Defined the objectives and scope of **CaloryInsight**.

Identified core features such as calorie tracking, BMI calculator, workout monitoring, and diet guidance.

Selected tools, technologies, and development timeline.

2.1.2. Requirement Analysis

Gathered user requirements and expectations for a fitness application.

Studied existing applications to identify limitations and improvement areas.

Determined functional and non-functional requirements.

2.1.3. Designing

Created system architecture, database design, and workflow diagrams.

Designed an attractive and user-friendly interface for smooth navigation.

Planned dashboard layout for health insights and tracking features.

2.1.4. Implementation

Developed frontend and backend modules of the project.

Implemented login/register system, BMI calculator, calorie tracker, and workout modules.

Integrated database for storing user data securely.

2.1.5. Testing

Tested each module individually and as a complete system.

Fixed bugs, improved performance, and ensured responsive design.

Verified accuracy of calculations and data handling.

2.1.6. Deployment

Deployed the final version of **CaloryInsight** on github for user access.

Ensured the application runs smoothly in the target environment.

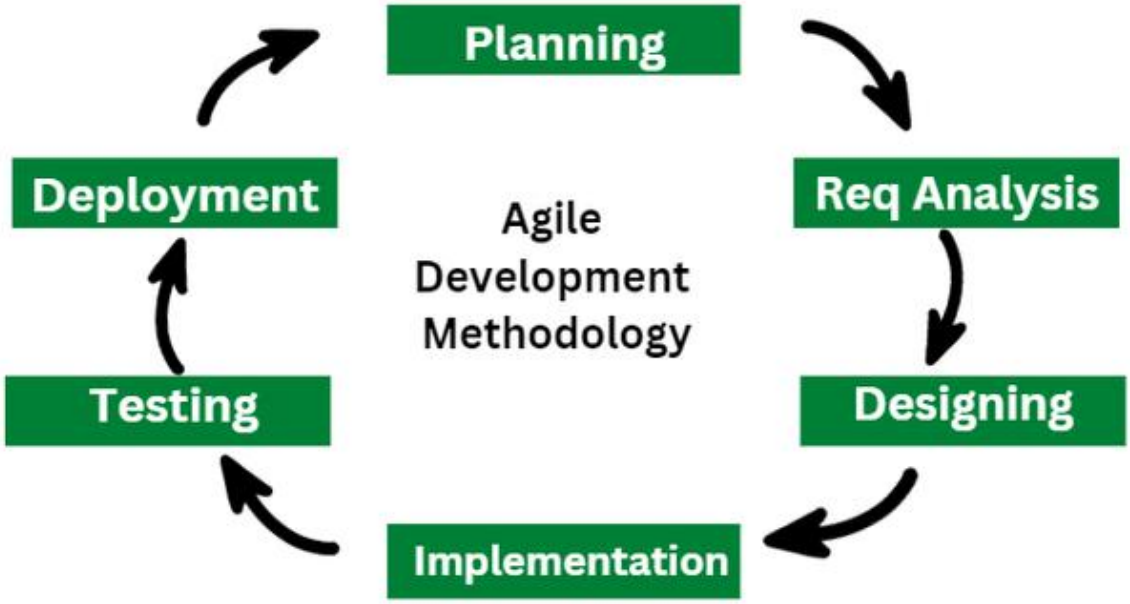


Figure 2.1

3. Tools and Technologies

3.1. Frontend Technologies

- React.js was used to develop the user interface of CaloryInsight. It provides a dynamic, responsive, and component-based structure for building interactive web pages.
- HTML5 was used for creating the structure of web pages.
- CSS3 was used for styling and designing the application interface.
- JavaScript (ES6) was used for implementing frontend logic and interactivity.

3.2. Backend Technologies

- Node.js was used as the runtime environment to execute server-side code efficiently.
- Express.js was used to build RESTful APIs and handle routing, requests, and responses between frontend and backend.

3.3. Database Technology

- MongoDB was used as the NoSQL database to store user details, calorie records, BMI data, and workout information.
- Mongoose was used as an Object Data Modelling (ODM) library to interact with MongoDB easily.

3.4. Authentication and Security

- JWT (JSON Web Token) was used for secure user authentication and authorization.
- bcrypt.js was used for password encryption and secure storage of user credentials.

3.5. Development Tools

- Visual Studio Code was used as the code editor for writing and managing project files.
- Git was used for version control and project management.
- GitHub was used for source code hosting and collaboration.
- Postman was used for API testing and debugging.

3.6. Deployment Tools

- Render / Vercel / Netlify can be used for deploying the frontend and backend of the project online.

4. Implementation

- The implementation of **CaloryInsight** was carried out using the MERN Stack, which includes MongoDB, Express.js, React.js, and Node.js. The project was divided into frontend, backend, and database modules to ensure smooth development and proper management of functionalities.
- The frontend of the application was developed using React.js to create an interactive and responsive user interface. Different components were created for features such as user registration, login, dashboard, BMI calculator, calorie tracker, and workout monitoring. CSS was used to design an attractive and user-friendly layout.
- The backend was developed using Node.js and Express.js. RESTful APIs were created to handle user requests, authentication, calorie calculations, BMI data processing, and communication with the database. Secure authentication was implemented using JWT tokens and password encryption techniques.
- The database was managed using MongoDB, where user details, health records, workout data, and calorie history were stored. Mongoose was used for schema creation and database operations.
- During implementation, all modules were integrated and tested to ensure proper functionality. The application was then deployed successfully, making it accessible for users to track their health data through a single platform.

5. System Design and Architecture

- The system design of **CaloryInsight** follows a three-tier architecture consisting of the presentation layer, application layer, and database layer. This architecture ensures better scalability, maintainability, and smooth communication between different modules of the application.
- The presentation layer represents the frontend of the system, which was developed using React.js. It provides an interactive and responsive user interface where users can register, log in, calculate BMI, track calories, monitor workouts, and view progress reports. The frontend communicates with the backend through API requests.

- The application layer represents the backend of the system, developed using Node.js and Express.js. It handles business logic, user authentication, request processing, calorie and BMI calculations, and data validation. RESTful APIs were created to manage communication between the frontend and database securely and efficiently.
- The database layer uses MongoDB to store and manage application data such as user profiles, calorie records, workout history, and health details. Mongoose was used to define schemas and perform database operations effectively.
- The overall architecture ensures secure data flow, modular development, and efficient performance. It also supports future enhancements such as AI-based recommendations, wearable device integration, and advanced analytics features.

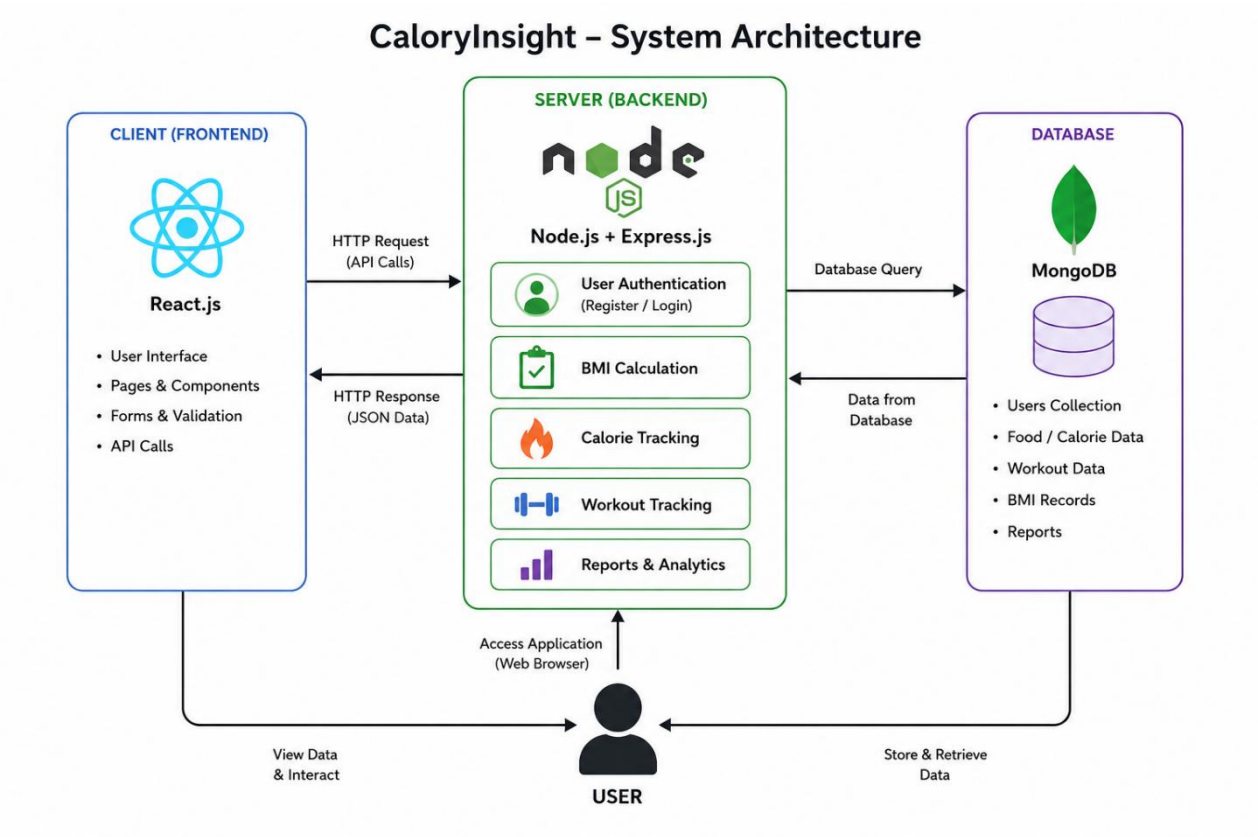


Figure 5.1

6. Workflow

6.1. User Registration and Login

- The user first creates an account by providing required details such as name, email, and password.
- After successful registration, the user logs into the system securely.

6.2. User Profile Setup

- The user enters personal details such as age, gender, height, and weight.
- This information is used for personalized calorie and BMI calculations.

6.3. Dashboard Access

- After login, the user is redirected to the dashboard.
- The dashboard displays all major features in one place.

6.4. BMI Calculation

- The user enters height and weight details.
- The system calculates BMI and displays the health category.

6.5. Calorie Tracking

- The user enters food intake or activity details.
- The system calculates calories consumed and calories burned.

6.6. Workout Monitoring

- The user selects exercises or workout duration.
- The system tracks workout activities and updates fitness progress.

6.7. Data Storage

- All user records such as profile data, BMI history, calorie logs, and workouts are stored securely in MongoDB.

6.8. Reports and Progress Analysis

- The system generates progress reports and displays health insights through the dashboard.

6.9. Logout

- After completing activities, the user can safely log out of the application.

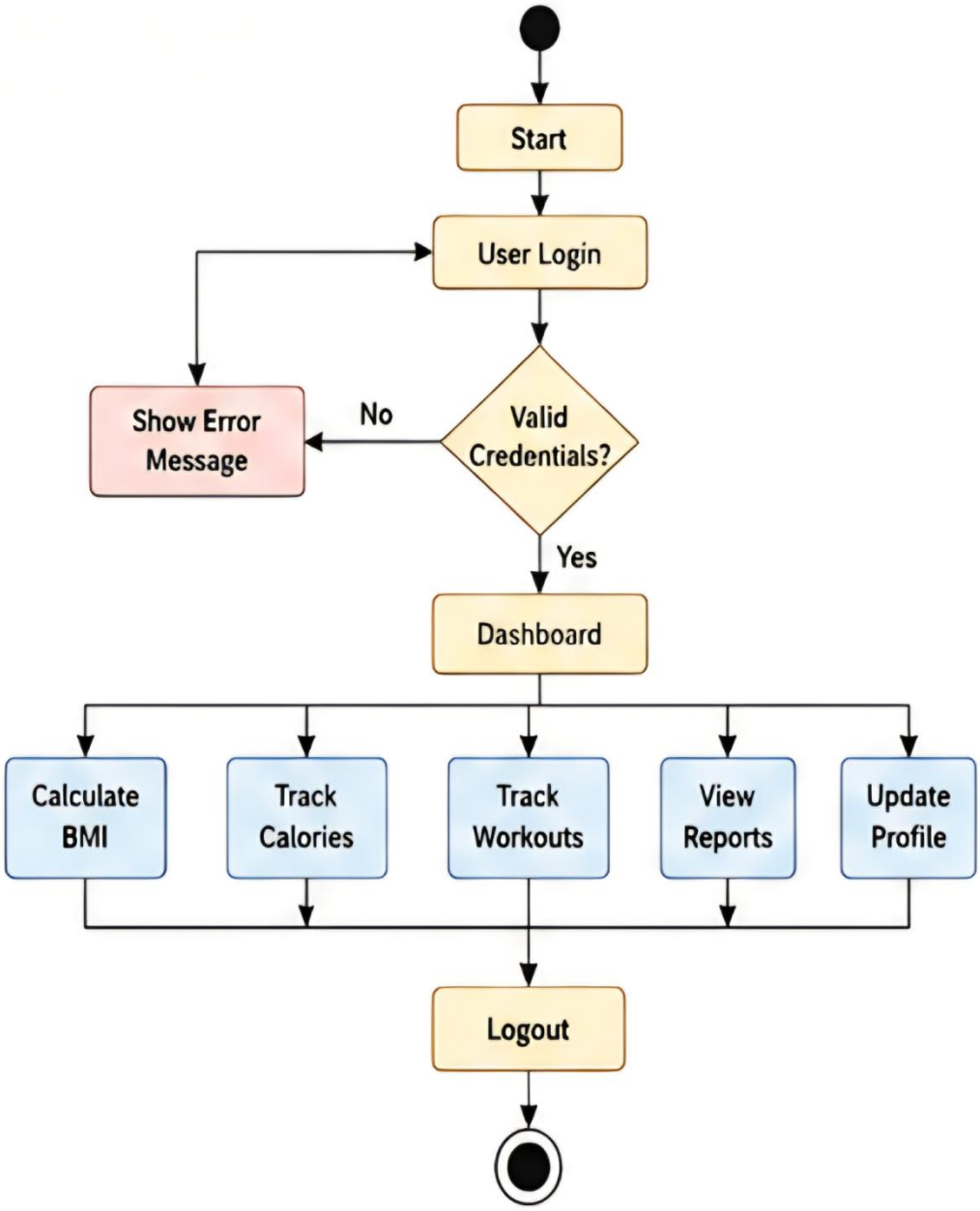

Start
User Login
Show Error Message
No
Valid Credentials?
Yes
Dashboard
Calculate BMI
Track Calories
Track Workouts
View Reports
Update Profile
Logout

Figure 6.1

7. Project Setup Guide

7.1. Clone the Repository

- git clone <your-repo-link>
- cd CaloryInsight

7.2. Backend Setup

- cd backend
- npm install
- Create a .env file in the backend folder and add:
- MONGO_URI=your_mongodb_connection_string
- JWT_SECRET=your_secret_key
- PORT=5000
- Start the backend server:
- npm start

Backend runs at:
http://localhost:5000

7.3. Frontend Setup

- Open a new terminal and run:
- cd frontend
- npm install
- npm run dev

Frontend runs at:
http://localhost:3000

7.4. Run the Project

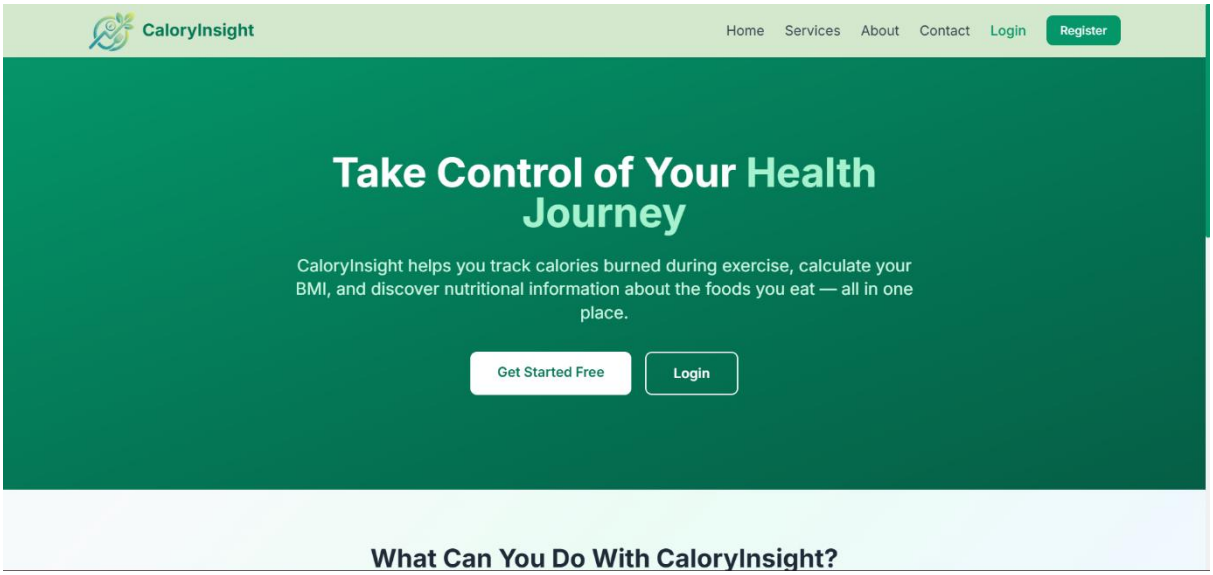
- Start backend first
- Then start frontend
- Open in browser:
  http://localhost:3000

8. Major Finding and Outcomes

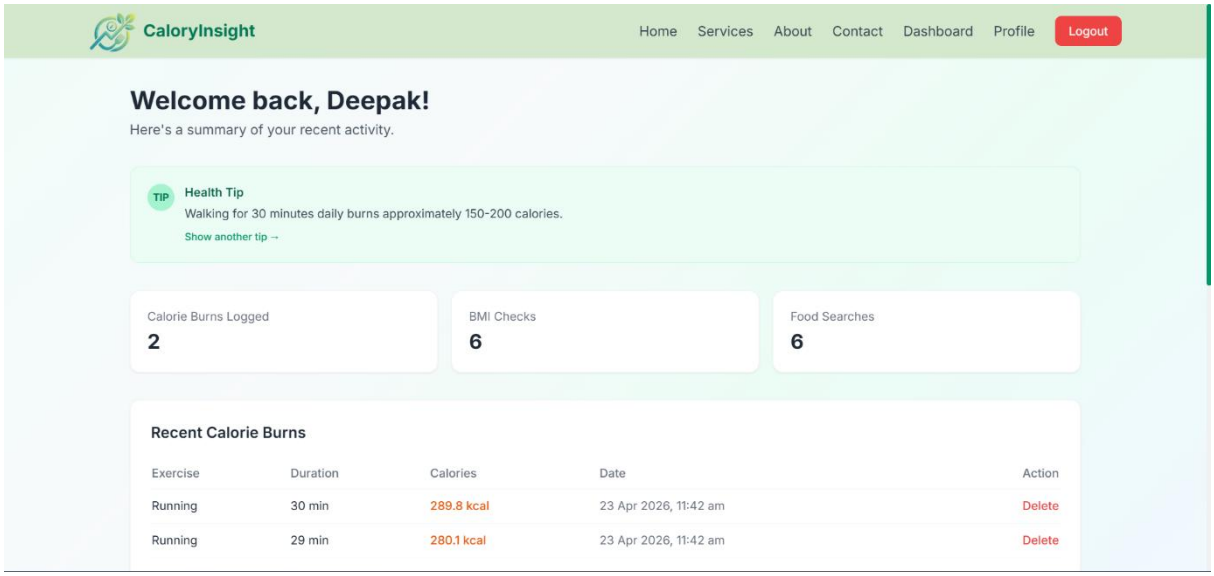
- Successfully developed a complete fitness web application using the MERN stack.
- Integrated calorie tracking, BMI calculation, and workout monitoring in one platform.
- Improved user convenience by reducing dependency on multiple applications.
- Provided accurate and quick health calculations based on user input.
- Ensured secure user authentication and data protection.
- Stored user records efficiently using MongoDB database.
- Delivered a responsive and interactive user interface using React.js.
- Generated progress reports and health insights for better decision-making.
- Reduced cost barriers by providing essential fitness tools digitally.

9. Screenshots and Code of Project

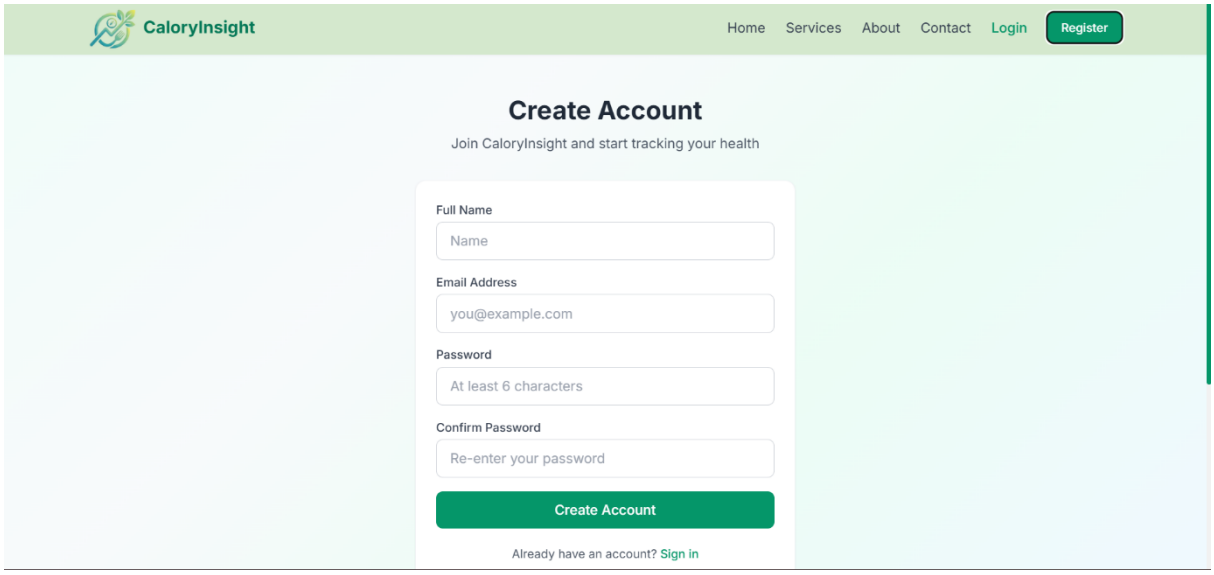
9.1. Home Page



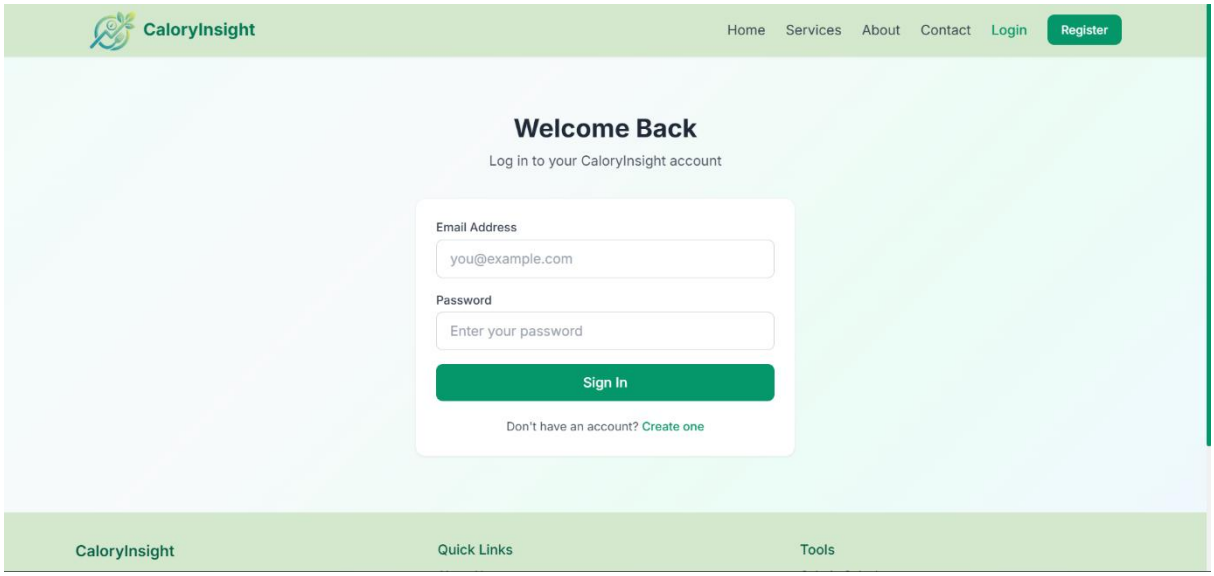
9.2. Dashboard Page



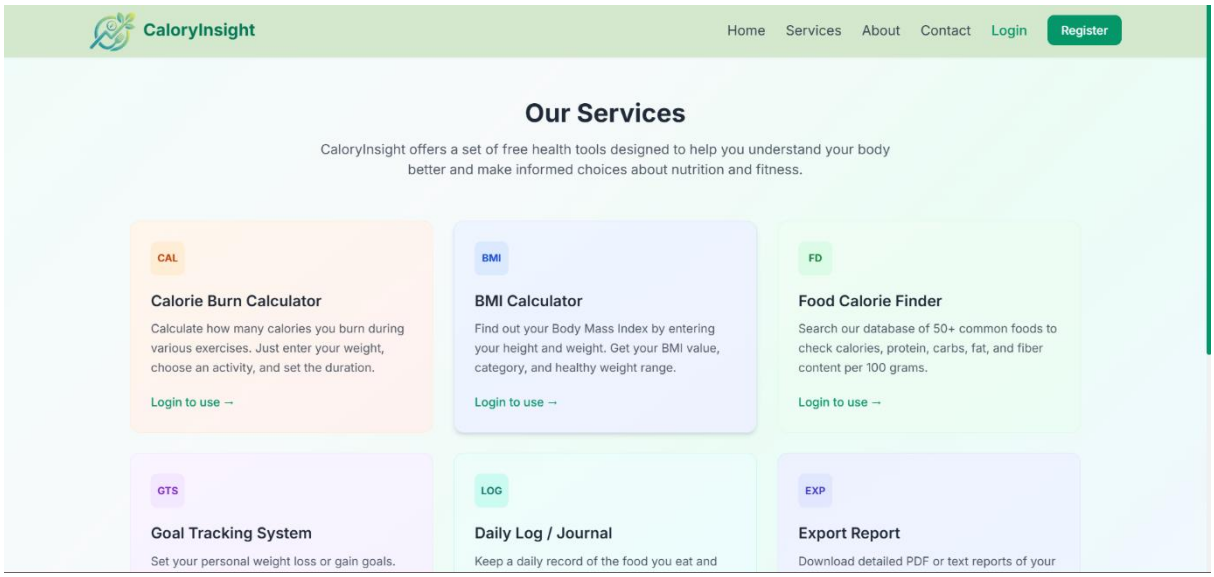
9.3. Registration Page



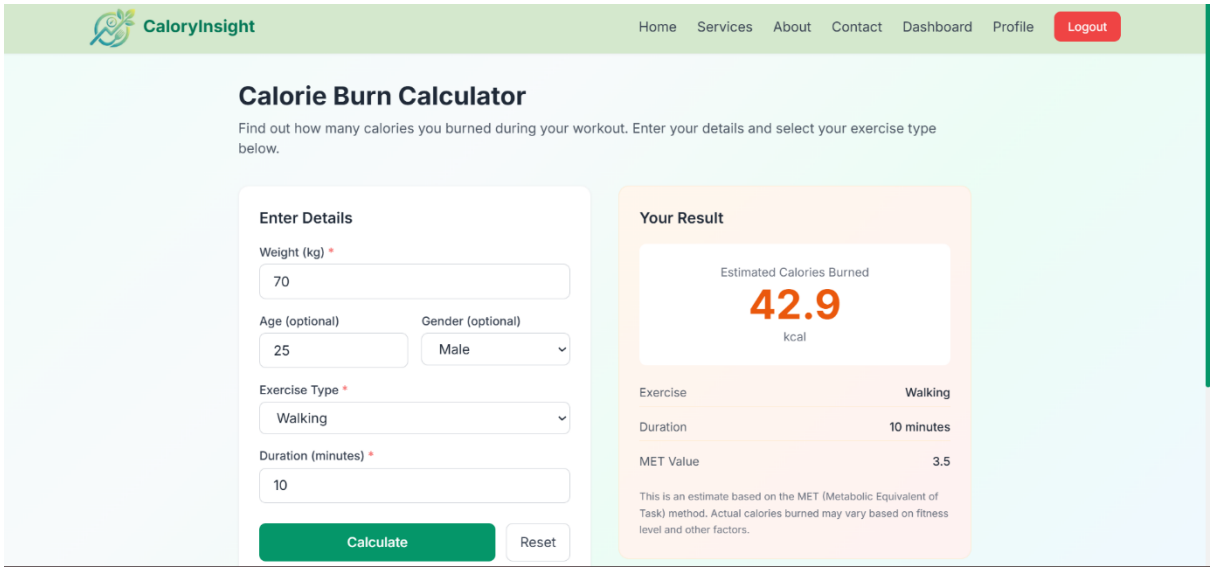
9.4. Login Page



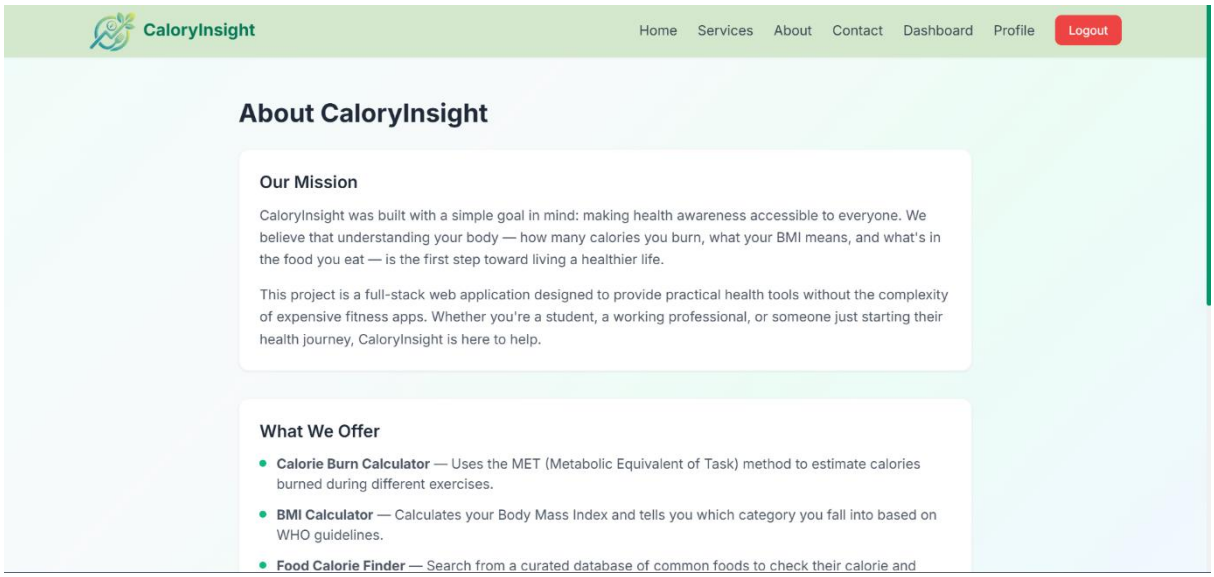
9.5. Service Page



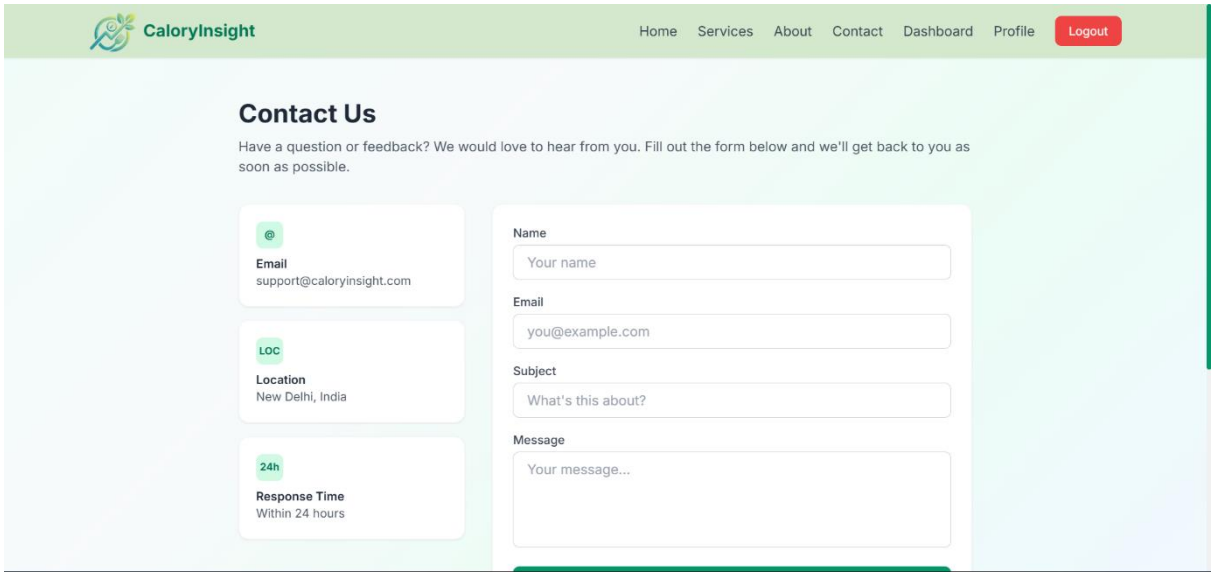
9.6. Calorie Burn Page



9.7. About Page



9.8. Contact Page



9.9. server.js file code

CaloryInsight-(Source-Code) > backend > JS server.js > ...

```javascript
import express from 'express';
import cors from 'cors';
import dotenv from 'dotenv';
import connectDB from './config/db.js';
import errorHandler from './middleware/errorHandler.js';
import dns from 'dns';


//Change DNS
dns.setServers(['1.1.1.1', '8.8.8.8']);

// load env variables before anything else
dotenv.config();

// connect database
connectDB();

const app = express();

// middleware setup
app.use(cors());
app.use(express.json());

import authRoutes from './routes/authRoutes.js';
import calculatorRoutes from './routes/calculatorRoutes.js';
import userRoutes from './routes/userRoutes.js';
import goalRoutes from './routes/goalRoutes.js';
import journalRoutes from './routes/journalRoutes.js';

// mount routes
app.use('/api/auth', authRoutes);
app.use('/api/calc', calculatorRoutes);
```

```
app.use('/api/user', userRoutes);
app.use('/api/goals', goalRoutes);
app.use('/api/journal', journalRoutes);

// quick health check route
app.get('/', (req, res) => {
  res.json({ message: 'CaloryInsight API is running' });
});

// global error handler - must be after routes
app.use(errorHandler);

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});

```

9.10. db.js file code

```
import mongoose from 'mongoose';

// connect to mongodb and return the connection
const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Database connection failed: ${error.message}`);
    process.exit(1);
  }
};

export default connectDB;

```

9.11. authController.js file code

```
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import { validationResult } from 'express-validator';
import User from '../models/User.js';

// helper to create a signed jwt
const generateToken = (userId) => {
  return jwt.sign({ id: userId }, process.env.JWT_SECRET, {
    expiresIn: '7d'
  });
};

// @route   POST /api/auth/register
// @desc    Create a new user account
const register = async (req, res, next) => {
  try {
    // check for validation errors from express-validator
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({ message: errors.array()[0].msg });
    }

    const { name, email, password } = req.body;

    // make sure email isn't taken
    const existingUser = await User.findOne({ email });
    if (existingUser) {
      return res.status(400).json({ message: 'An account with that email already exists' });
    }

    // hash password with bcrypt
    const salt = await bcrypt.genSalt(10);
```

9.12. main.jsx file code

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App';
import { AuthProvider } from './context/AuthContext';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <AuthProvider>
        <App />
      </AuthProvider>
    </BrowserRouter>
  </React.StrictMode>
);

```

9.13. App.jsx file code

```jsx
import { Routes, Route } from 'react-router-dom';
import Navbar from './components/Navbar';
import Footer from './components/Footer';
import ProtectedRoute from './components/ProtectedRoute';

// pages
import Home from './pages/Home';
import About from './pages/About';
import Services from './pages/Services';
import Contact from './pages/Contact';
import Login from './pages/Login';
import Register from './pages/Register';
import Dashboard from './pages/Dashboard';
import Profile from './pages/Profile';
import CalorieBurn from './pages/CalorieBurn';
import BmiCalculator from './pages/BmiCalculator';
import FoodCalories from './pages/FoodCalories';
import GoalTracking from './pages/GoalTracking';
import DailyJournal from './pages/DailyJournal';
import ExportReport from './pages/ExportReport';

function App() {
  return (
    <div className="flex flex-col min-h-screen">
      <Navbar />

      <main className="flex-1">
        <Routes>
          {/* public routes */}
          <Route path="/" element={<Home />} />
          <Route path="/about" element={<About />} />
          <Route path="/services" element={<Services />} />
```

```
33            <Route path="/contact" element={<Contact />} />
34            <Route path="/login" element={<Login />} />
35            <Route path="/register" element={<Register />} />
36
37            {/* protected routes - require login */}
38            <Route path="/dashboard" element={
39              <ProtectedRoute><Dashboard /></ProtectedRoute>
40            } />
41            <Route path="/profile" element={
42              <ProtectedRoute><Profile /></ProtectedRoute>
43            } />
44            <Route path="/calorie-burn" element={
45              <ProtectedRoute><CalorieBurn /></ProtectedRoute>
46            } />
47            <Route path="/bmi" element={
48              <ProtectedRoute><BmiCalculator /></ProtectedRoute>
49            } />
50            <Route path="/food-search" element={
51              <ProtectedRoute><FoodCalories /></ProtectedRoute>
52            } />
53            <Route path="/goals" element={
54              <ProtectedRoute><GoalTracking /></ProtectedRoute>
55            } />
56            <Route path="/journal" element={
57              <ProtectedRoute><DailyJournal /></ProtectedRoute>
58            } />
59            <Route path="/export" element={
60              <ProtectedRoute><ExportReport /></ProtectedRoute>
61            } />
62
```

```
63            {/* 404 fallback */}
64            <Route path="*" element={
65              <div className="flex flex-col items-center justify-center min-h-[60vh] text-center px-4">
66                <h1 className="text-6xl font-bold text-gray-300 mb-4">404</h1>
67                <p className="text-gray-600 mb-4">Page not found</p>
68                <a href="/" className="text-primary-600 hover:text-primary-700 font-medium">
69                  Go back home
70                </a>
71              </div>
72            } />
73          </Routes>
74        </main>
75
76        <Footer />
77      </div>
78    );
79  }
80
81  export default App;
82
```

10. Conclusion and Future Scope

10.1. Conclusion

The development of CaloryInsight successfully achieved the objective of creating a smart and user-friendly fitness web application using the MERN stack. The system integrates important health features such as calorie tracking, BMI calculation, workout monitoring, and progress analysis into a single platform. It reduces the need for multiple applications and provides users with a convenient way to manage their health and fitness activities.

The project also ensures secure data storage, responsive design, and smooth user experience. By combining technology with health management, CaloryInsight offers an affordable and effective solution for users who want to maintain a healthy lifestyle. Overall, the project demonstrates the practical use of web technologies in solving real-world fitness challenges.

10.2. Future Scope

The future scope of CaloryInsight includes the integration of Artificial Intelligence to provide personalized diet plans, workout suggestions, and health recommendations based on user data. This can make the application smarter and more effective for individual users.

The project can also be enhanced by connecting wearable fitness devices such as smartwatches and fitness bands for real-time tracking of heart rate, steps, calories burned, and sleep patterns.

Additional future improvements may include advanced analytics, goal-based progress tracking, social sharing features, multilingual support, mobile application development, and cloud deployment for wider accessibility. These enhancements can transform CaloryInsight into a complete digital health companion.

11. References

- React.js Official Documentation – https://react.dev/
- Node.js Documentation – https://nodejs.org/
- Express.js Documentation – https://expressjs.com/
- MongoDB Documentation – https://www.mongodb.com/docs/
- Tailwind CSS Documentation – https://tailwindcss.com/docs
- Axios Documentation – https://axios-http.com/
- Body Mass Index Reference – https://www.cdc.gov/bmi/
- Calorie Reference – https://www.nhs.uk/live-well/healthy-weight/calories/

12. Appendices

- Operating System: Windows / Linux / macOS
- Frontend: React.js, HTML5, CSS3, JavaScript
- Backend: Node.js, Express.js
- Database: MongoDB
- Code Editor: Visual Studio Code
- API Testing Tool: Postman
- User Registration and Login
- Dashboard and Reports
- User Profile Management